



Bericht in

Entwicklung einer Metaheuristik für das Job Shop Scheduling Problem unter Berücksichtigung von reihenfolgeabhängigen Rüstzeiten

Jakob Weber Matr.-Nr. 3869088
Johannes Walz Matr.-Nr. 3533824

Prüfer: Prof. Dr. David Müller

© Jakob Weber Matr.-Nr. 3869088, Johannes Walz Matr.-Nr. 3533824, 2026

Diese Arbeit, einschließlich ihrer Teile, **ist urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ohne Zustimmung von den Urhebern ist untersagt und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmung und die Speicherung und Verarbeitung in elektronischen Systemen.

Inhaltsverzeichnis

1	Einleitung	2
1.1	Problemstellung	2
1.2	Zielsetzung	2
1.3	Aufbau der Arbeit	2
2	Notation	3
3	Methodik	5
3.1	Eröffnungsheuristik - Giffler-Thompson-Algorithmus	5
3.2	Metaheuristik - Tabu-Suche und Nachbarschaftsdefinitionen	5
3.3	Kritischer Pfad und kritische Blöcke	6
3.4	Nachbarschaftserzeugung	6
3.5	Hardwarespezifikationen	6
3.6	Forschungsdesign	7
3.7	Implementierung	7
4	Ergebnisse	8
5	Diskussion	9

Kapitel 1

Einleitung

1.1 Problemstellung

1.2 Zielsetzung

Wie beeinflusst die Wahl der Nachbarschaftsdefinition in einer Tabu Suche den Makespan von Job-Shop-Scheduling-Problemen mit reihenfolgeabhängigen Rüstzeiten?

1.3 Aufbau der Arbeit

Kapitel 2

Notation

Notation der Job-Shop-Scheduling-Problems mit sequenzabhängigen Rüstzeiten

In diesem Kapitel wird die Notation für das betrachtete Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten eingeführt. Die Notation dient als Grundlage für die Beschreibung der Tabu-Search-Heuristik und der verwendeten Bewertungslogik.

Gegeben ist eine Menge von Jobs J und eine Menge von Maschinen M . Jeder Job $j \in J$ besteht aus einer geordneten Folge von Operationen

$$O_j = (o_{j,1}, o_{j,2}, \dots, o_{j,q_j}).$$

Die Reihenfolge der Operationen innerhalb eines Jobs ist vorgegeben. Eine Operation $o_{j,k}$ darf daher erst beginnen, wenn ihre direkte Vorgängeroperation $o_{j,k-1}$ abgeschlossen ist.

Jede Operation $o_{j,k}$ wird auf genau einer Maschine $m_{j,k} \in M$ bearbeitet und besitzt eine Bearbeitungszeit $p_{j,k}$. Zwischen zwei Operationen, die auf derselben Maschine direkt aufeinanderfolgen, kann eine sequenzabhängige Rüstzeit $s_{u,v}$ anfallen. Diese hängt von der vorherigen Operation u und der nachfolgenden Operation v ab.

Tabelle 2.1: Notation des Job-Shop-Scheduling-Problems

Symbol	Bedeutung
J	Menge der Jobs
j	Index eines Jobs, $j \in J$
M	Menge der Maschinen
m	Index einer Maschine, $m \in M$
O_j	geordnete Menge der Operationen von Job j
$o_{j,k}$	k -te Operation von Job j
q_j	Anzahl der Operationen von Job j
O	Menge aller Operationen
$m_{j,k}$	Maschine, auf der Operation $o_{j,k}$ bearbeitet wird
$p_{j,k}$	Bearbeitungszeit der Operation $o_{j,k}$
$s_{u,v}$	Rüstzeit zwischen zwei direkt aufeinanderfolgenden Operationen u und v
$t(o)$	Startzeit der Operation o
$C(o)$	Endzeit der Operation o
$C_{\max}$	Makespan einer Lösung
$F(S)$	Zielfunktionswert der Lösung S

Notation der Tabu-Suche

Zur Lösung des Problems wird eine Tabu-Search-Heuristik verwendet. Die aktuelle Lösung wird mit S bezeichnet. Die beste bisher gefundene Lösung wird als S^* notiert. Die Nachbarschaft der aktuellen Lösung wird durch $NB(S)$ beschrieben. Sie enthält alle Lösungen, die durch zulässige lokale Bewegungen aus S erzeugt werden können.

Die Tabu-Liste TL speichert Bewegungsattribute, die für eine begrenzte Anzahl von Iterationen tabu sind. Dadurch wird ein unmittelbares Zurückkehren in vorherige Lösungen verhindert. Tabuierte Bewegungen können durch das Aspirationskriterium zugelassen werden, wenn sie zu einer neuen globalen Bestlösung führen.

Tabelle 2.2: Notation der Tabu-Search-Heuristik

Symbol	Bedeutung
$\hat{S}$	erzeugte Startlösung
S	aktuelle Lösung
S^*	beste bisher gefundene Lösung
S'	Nachbarlösung der aktuellen Lösung
$NB(S)$	Nachbarschaft der aktuellen Lösung S
TL	Tabu-Liste
TL^{Dauer}	Gültigkeitsdauer eines Tabu-Eintrags
I	Anzahl der Iterationen ohne Verbesserung
I^{max}	maximale Anzahl an Iterationen ohne Verbesserung
I_{total}	Gesamtzahl der Iterationen
$I_{\text{total}}^{\text{max}}$	maximale Gesamtzahl an Iterationen
$I^{\text{Diversifikation}}$	Schwelle für zusätzliche Diversifikationsbewegungen
τ^{max}	optionales Zeitlimit
$F(S)$	Zielfunktionswert der Lösung S

Da der Makespan minimiert wird, gilt äquivalent:

$$C_{\max}(S') < C_{\max}(S^*).$$

Kapitel 3

Methodik

3.1 Eröffnungsheuristik - Giffler-Thompson-Algorithmus

3.2 Metaheuristik - Tabu-Suche und Nachbarschaftsdefinitionen

Algorithm 1 Tabu Search für das Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten

```
1:  $S \leftarrow \hat{S}$ 
2:  $S^* \leftarrow S$ 
3:  $TL \leftarrow \emptyset$ 
4:  $I_{\text{total}} \leftarrow 0$ 
5:  $I \leftarrow 0$ 
6:  $\tau \leftarrow 0$ 
7: while  $I_{\text{total}} < I_{\text{total}}^{\max}$  and  $I < I^{\max}$  and  $\tau < \tau^{\max}$  do
8:    $I_{\text{total}} \leftarrow I_{\text{total}} + 1$ 
9:   Erzeuge die Nachbarschaft  $NB(S)$  der aktuellen Lösung
10:  if  $I \geq I^{\text{div}}$  then
11:    Ergänze zufällige nicht-kritische Bewegungen zu  $NB(S)$ 
12:  end if
13:  for  $S' \in NB(S)$  do (jede Nachbarlösung)
14:    if  $S'$  ist unzulässig or (tabuiert and Aspiration nicht erfüllt) then
15:      Entferne  $S'$  aus  $NB(S)$ 
16:    end if
17:  end for
18:  Wähle die beste zulässige Nachbarlösung  $S' \in NB(S)$ 
19:   $S \leftarrow S'$ 
20:  Entferne abgelaufene Einträge aus  $TL$ 
21:   $TL \leftarrow TL \cup \{\text{inverse Bewegung von } S'\}$ 
22:  if  $F(S) < F(S^*)$  then
23:     $S^* \leftarrow S$ 
24:     $I \leftarrow 0$ 
25:  else
26:     $I \leftarrow I + 1$ 
27:  end if
28: end while
29: return  $S^*$ 
```

Die Suche beginnt mit der Erzeugung einer zulässigen Startlösung (S). Anschließend wird die beste gefundene Lösung mit ($\hat{S} = S$) initialisiert. Danach werden die Tabu-Liste (TL), der Gesamtiterationenzähler (I^{total}) und der Zähler für Iterationen ohne Verbesserung (I) initialisiert. Die Suche wird fortgeführt, bis ein Abbruchkriterium erfüllt ist. Der Gesamtiterationenzähler wird in jedem Schleifendurchlauf um eins erhöht.

Der Algorithmus verwendet drei Abbruchkriterien. Erstens endet die Suche nach Erreichen der maximalen Gesamtzahl an Iterationen. Zweitens endet sie nach einer festgelegten Anzahl an Iterationen ohne Verbesserung. Drittens kann ein Zeitlimit gesetzt werden. Der Timer wird nur aktiviert, wenn beim Aufruf des Algorithmus ein Zeitlimit übergeben wird.

In jeder Iteration wird zunächst die Nachbarschaft ($NB(S)$) der aktuellen Lösung erzeugt. Ergänzend werden potenzielle Kandidatenbewegungen gebildet. Wird über mehrere Iterationen keine Verbesserung erzielt, werden zusätzliche zufällige nicht-kritische Bewegungen ergänzt. Diese dienen der Diversifikation der Suche. Liegen mehrere zulässige Kandidaten mit identischem Makespan vor, werden diese zufällig durchmischt. Dadurch soll ein Festlaufen in lokalen Optima reduziert werden.

Tabuierte Bewegungen werden verworfen, sofern die Aspirationsbedingung nicht erfüllt ist. Diese ist erfüllt, wenn eine tabuierte Bewegung zu einer Verbesserung der besten bisher gefundenen Lösung führt. Aus den verbleibenden Kandidaten wird die beste zulässige Nachbarlösung (S') ausgewählt. Existiert keine zulässige Nachbarlösung, wird die Suche beendet.

Wird eine Nachbarlösung akzeptiert, wird die aktuelle Lösung durch (S') ersetzt. Anschließend wird die Tabu-Liste mit der inversen Bewegung aktualisiert. Dadurch wird ein direktes Zurückkehren in den vorherigen Zustand verhindert. Ist die neue Lösung besser als die bisher beste Lösung, gilt also ($F(S) < F(S^*)$), wird (S^*) durch (S) ersetzt und der Verbesserungszähler zurückgesetzt. Andernfalls wird der Verbesserungszähler erhöht. Nach Abschluss der Suche wird die beste gefundene Lösung (S^*) zurückgegeben.

3.3 Kritischer Pfad und kritische Blöcke

Um den kritischen Pfad zu bestimmen, wird für jede Operation die Startzeit des Vorgängers gespeichert. Ausgehend von der Operation mit der größten Endzeit wird dieser Vorgängerpfad rückwärts verfolgt. Dadurch entsteht ein Pfad von Operationen, der den aktuellen Makespan bestimmt. Aus dem kritischen Pfad werden daraufhin kritische Blöcke extrahiert. Ein kritischer Block ist eine maximale Folge von Operationen, die auf dem kritischen Pfad liegen, auf derselben Maschine bearbeitet werden und dort direkt aufeinanderfolgen.

3.4 Nachbarschaftserzeugung

Die Nachbarschaft ($NB(S)$) besteht aus lokalen Veränderungen der Maschinenreihenfolgen. Im Mittelpunkt stehen Bewegungen innerhalb kritischer Blöcke. Dadurch konzentriert sich die Suche auf Bereiche, die den aktuellen Makespan beeinflussen.

In der Implementierung sind drei Nachbarschaftsvarianten vorgesehen. Die Adjacent-Critical-Nachbarschaft vertauscht benachbarte Operationen innerhalb eines kritischen Blocks. Die N5-Nachbarschaft vertauscht die ersten beiden und die letzten beiden Operationen eines kritischen Blocks. Die N6-Nachbarschaft erweitert N5 um Insert-Bewegungen. Dabei werden innere Operationen eines kritischen Blocks an den Blockanfang oder an das Blockende verschoben.

3.5 Hardwarespezifikationen

Die Instanzen wurden auf einem Apple MacBook Pro aus dem Jahr 2023 ausgeführt. Das System ist mit einem Apple M3 Pro Prozessor sowie 18 GB gemeinsam genutztem Arbeitsspeicher ausgestattet.

Da es sich beim M3 Pro um ein System-on-a-Chip handelt, sind CPU, GPU und Arbeitsspeicher eng integriert. Die Berechnungen wurden lokal auf dem Gerät durchgeführt, ohne zusätzliche externe Rechenressourcen oder Serverinfrastruktur zu verwenden.

Tabelle 1: Ergebnisse der Giffler-Thompson-Prioritätsregeln

- Prioritätsregel- Cmax- Laufzeit- Anzahl Rüstvorgänge oder gesamte Rüstzeit, falls relevant

Tabelle 2: Vergleich der Tabu-Search-Nachbarschaften

- Nachbarschaft- bester Cmax - durchschnittlicher Cmax- schlechtester Cmax - Standardabweichung- durchschnittliche Laufzeit- Anzahl Iterationen

Tabelle 3: Vergleich mit OR-Tools

- Instanz - OR-Tools Cmax - Tabu Search Cmax - absolute Abweichung - relative Abweichung in % - Laufzeit OR-Tools - Laufzeit Tabu Search

3.6 Forschungsdesign

3.7 Implementierung

In this chapter, after the ?? chapter we're actually using some code!

```

1     x = 1
2     if x == 1:
3         # indented four spaces
4         print("x is 1.")

```

Listing 1: This is an example of inline listing

You can also include listings from a file directly:

```

1 x = 1
2 if x == 1:
3     # indented four spaces
4     print("x is 1.")

```

Listing 2: This is an example of included listing

Kapitel 4

Ergebnisse

Kapitel 5

Diskussion

Abbildungsverzeichnis

Quellcodeverzeichnis

1	This is an example of inline listing	7
2	This is an example of included listing	7

Tabellenverzeichnis

2.1	Notation des Job-Shop-Scheduling-Problems	3
2.2	Notation der Tabu-Search-Heuristik	4

Literaturverzeichnis